

Transformers: From Intuition to PyTorch

M.I. Capel

ETS Ingenierías Informática y
Telecomunicación
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Granada
Email: manuelcapel@ugr.es
<http://lsi2.ugr.es/mcapel/>

Febrero 26, 2026
Máster Universitario en Desarrollo de Software



- 1 Tokens, embeddings, and the problem
- 2 Self-attention as weighted sums
- 3 Q/K/V + scaling
- 4 Training

Sequence modeling: what we want

Given a sequence of tokens:

$$(w_1, \dots, w_t)$$

we often want:

- **Sequence labeling:** output one label per token, e.g.,
Named Entity Recognition (NER)
- **Sequence classification:** output one label for the whole
sequence
- **Autoregressive generation:** predict next token repeatedly

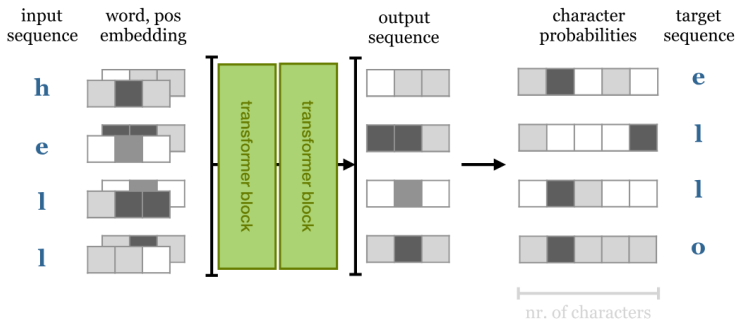
Embedding layer

Map each token id to a vector:

$$w_i \mapsto x_i \in \mathbb{R}^k$$

An embedding layer is a matrix:

$$E \in \mathbb{R}^{|V| \times k}, \quad x_i = E[w_i]$$



Why do we need positional information?

Self-attention by itself is permutation-equivariant:

$$(x_1, x_2, \dots, x_t) \rightarrow (y_1, y_2, \dots, y_t)$$

If we permute inputs, outputs permute the same way.

Transformers add **positional encodings** (learned or fixed):

$$\tilde{x}_i = x_i + p_i$$

sequence of tokens: "Mary greets Susan"

Positional encoding **for** token Susan:
position i

0		(0.00, 0.0)
1		(0.00, 0.2)
2		(0.85, 0.0)

RNN intuition vs attention intuition

- RNN: build state left-to-right; hard to parallelize.

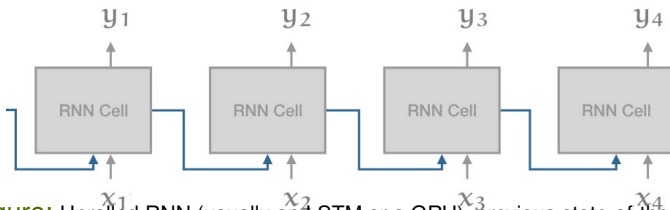


Figure: Unrolled RNN (usually an LSTM or a GRU), previous state-of-the-art architecture

- Attention: let each position look at all positions; highly parallel.

Self-attention operation

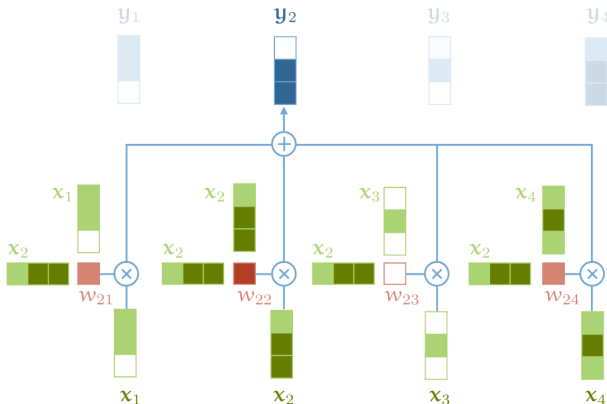


Figure: Representation of the fundamental operation of any transformer architecture

Key question of a self-attention architecture:

How does a token decide *which* other tokens matter?

Basic self-attention (idea)

For each position i , produce output as a weighted sum of inputs:

$$y_i = \sum_{j=1}^t w_{ij} x_j, \quad \sum_j w_{ij} = 1, \quad w_{ij} \geq 0$$

Weights come from similarity (dot products):

$$s_{ij} = x_i^\top x_j, \quad w_{ij} = \text{softmax}_j(s_{ij})$$

Softmax reminder

Given scores $s_1, \dots, s_t$:

$$\text{softmax}(s_j) = \frac{e^{s_j}}{\sum_{u=1}^t e^{s_u}}$$

Properties:

- turns arbitrary real scores into probabilities (nonnegative, sum to 1)
- only relative differences matter:
 $\text{softmax}(s) = \text{softmax}(s + c)$

Why dot products?

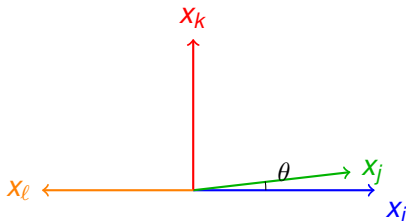
Dot product as alignment:

$$x_i^\top x_j = \|x_i\| \|x_j\| \cos(\theta)$$

If vectors point in similar directions, dot product is large.
Learning changes embeddings so “useful” pairs align for the task.

Dot Product as Alignment

$$x_i^\top x_j = \|x_i\| \|x_j\| \cos(\theta)$$



- **Aligned vectors (small θ)** $\rightarrow$ large positive dot product $\rightarrow$ similar
- **Orthogonal vectors (90°)** $\rightarrow$ dot product = 0 $\rightarrow$ **not similar**
- **Opposite direction (180°)** $\rightarrow$ negative dot product $\rightarrow$ **opposed meaning**

Key limitation of basic attention

In basic attention, the same vector x_i must:

- decide who to attend to (query role)
- be compared as a candidate (key role)
- provide content to copy (value role)

Modern transformers separate these roles.

Queries, Keys, Values

Introduce learned linear maps:

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$$

Compute scores:

$$s_{ij} = q_i^\top k_j$$

Weights:

$$w_{ij} = \text{softmax}_j \left(\frac{s_{ij}}{\sqrt{d}} \right)$$

Output:

$$y_i = \sum_{j=1}^t w_{ij} v_j$$

Toy example

"The cat walks on street":

- Let's take embeddings as 2D vectors

	words in sequence	vector
1		
2	the	(0.1, 0.0)
3	cat	(1.0, 0.9)
4	walks	(0.9, 1.0)
5	on	(0.0, 0.1)
6	street	(0.8, 0.7)

- Dot products with token 'cat':

$$s_{10} = v_{cat} \cdot v_{the} = (1.0, 0.9) \cdot (0.1, 0.0)^T = 0.1$$

$$s_{11} = v_{cat} \cdot v_{cat} = (1.0, 0.9) \cdot (1.0, 0.9)^T = 1.81$$

$$s_{12} = v_{cat} \cdot v_{walks} = (1.0, 0.9) \cdot (0.9, 1.0)^T = 1.80$$

$$s_{13} = v_{cat} \cdot v_{on} = (1.0, 0.9) \cdot (0.0, 0.1)^T = 0.09$$

$$s_{14} = v_{cat} \cdot v_{street} = (1.0, 0.9) \cdot (0.8, 0.7)^T = 1.43$$

Toy example-ii

$$\text{softmax}_u(s_{1,u}) = \text{softmax}_u(s_{cat,u}) = \left(\frac{e^{s_{cat,u}}}{\sum_j e^{s_{cat,j}}} \right)$$

$e^{\frac{s_{10}}{\sqrt{2}}}$	1.1073
$e^{\frac{s_{11}}{\sqrt{2}}}$	3.5962
$e^{\frac{s_{12}}{\sqrt{2}}}$	3.5709
$e^{\frac{s_{13}}{\sqrt{2}}}$	1.0657
$e^{\frac{s_{14}}{\sqrt{2}}}$	2.7488
Sum	12.0889

$\text{softmax}_0(\frac{s_{10}}{\sqrt{(2)}})$	0.0916
$\text{softmax}_1(\frac{s_{11}}{\sqrt{(2)}})$	0.2975
$\text{softmax}_2(\frac{s_{12}}{\sqrt{(2)}})$	0.2954
$\text{softmax}_3(\frac{s_{13}}{\sqrt{(2)}})$	0.0882
$\text{softmax}_4(\frac{s_{14}}{\sqrt{(2)}})$	0.2273
Sum	1.0000

$$w_{ij} = \text{softmax}_j \left(\frac{s_{ij}}{\sqrt{d}} \right)$$

Why scale by $\sqrt{d}$?

If $q, k \in \mathbb{R}^d$ have typical component size ~ 1 , then dot products grow with d .

Large scores saturate softmax $\Rightarrow$ small gradients.

To avoid this, it is necessary to scale: $\frac{q_i^\top k_j}{\sqrt{d}}$

Scaling keeps score magnitudes in a “healthier” range, i.e., values of order $\mathcal{O}(1)$: large enough to carry information, but not so large that they cause numerical problems. Without scaling:

- Large dimension $\Rightarrow$ very large dot products
- Softmax becomes extremely peaked
- Gradients collapse

Scaling keeps values around $\mathcal{O}(1)$.

Important reflections on the **attention** mechanism

There is no “human” interpretation for an embedding vector

$v_{\text{cat}} = (0.9, 1.0)$ **does not mean** that the token `cat` has value 0.9 on “feature 1”.

What does an embedding mean?

- If we represent tokens (words) as vectors in a space where dot products help solve a task, then the coordinates of the embedding are numerical values that make the task work.
- They do not correspond to interpretable semantic features.
- Infinitely many correct embeddings exist and the model behaves identically as long as dot products are preserved.

Important reflections on the **attention** mechanism – II

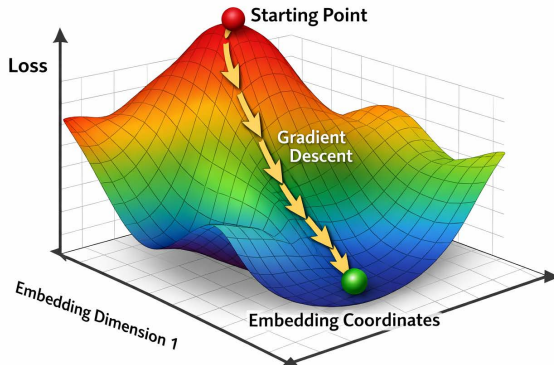


Figure: Vector optimization with gradient descent

Important reflections on the **attention** mechanism – III

Embeddings and geometric equivalence

- Embeddings are typically initialized at random.
- Training updates them through gradient descent.
- The loss function encourages dot products to become useful for the task.
- The coordinates settle into whatever values minimize the loss.
- Only geometric relationships (e.g., distances, angles, dot products) matter.
- Embedding coordinates are learned parameters, just like the rest of the model.

From Embeddings to Attention (single head)

For a sequence of length t , stack embeddings:

$$X \in \mathbb{R}^{t \times K}$$

Queries, Keys and Values are linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

with $W_Q, W_K, W_V \in \mathbb{R}^{K \times d}$.

Thus,

$$Q, K, V \in \mathbb{R}^{t \times d}.$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

The Embedding Layer

An embedding layer is a learnable matrix

$$E \in \mathbb{R}^{|V| \times K},$$

where $|V|$ is the vocabulary size and K is the embedding dimension.

The embedding of token t is the t -th row: $v_t = E_t \in \mathbb{R}^K$. E is initialized randomly and learned during training.

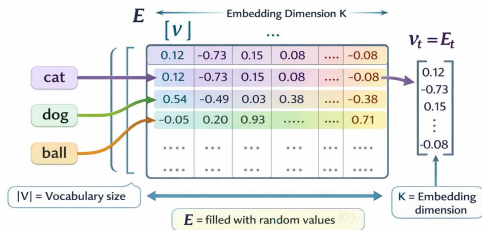


Figure: Embedding matrix as a learnable lookup table

Matrix form (single head)

For a sequence of length t :

$$Q \in \mathbb{R}^{t \times d_k}, \quad K \in \mathbb{R}^{t \times d_k}, \quad V \in \mathbb{R}^{t \times d_v}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Here d_k is the key/query dimension (per head), not the vocabulary embedding size.

Training process

Fundamental idea

We train tasks, not embeddings directly.

The fundamental pipeline

word indices $\Rightarrow$ embeddings lookup $\Rightarrow$ vectors $\Rightarrow$
self-attention $\Rightarrow$ output $\Rightarrow$ loss $\mathcal{L}(\cdot)$ calculus

Training process

- The model makes a prediction
- The prediction is initially wrong
- The loss gradient is computed
- Backpropagation computes:

$$\nabla_{\mathbf{v}_t} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{v}_t} \in \mathbb{R}^K$$

- All model parameters (including embeddings) are updated by gradient descent over many iterations.

Training process-II

Loss function and model's prediction

- There is a loss function $\mathcal{L}(\text{prediction}, \text{target})$ so that *lower loss $\Rightarrow$ better prediction*
- When using self-attention, dot products affect weights

$$w_{ij} = \text{softmax}\left(\frac{q_i \cdot k_j}{\sqrt{d_k}}\right)$$

$$y_i = \sum_j w_{ij} v_j$$

- The attention weights influence the output y_i , which affects the prediction and the loss:

$$v_{\text{cat}} \cdot v_{\text{walk}} \Rightarrow w_{ij} \Rightarrow y \Rightarrow \mathcal{L}$$

Training process – III

Gradient descent and the loss

Vectors are updated by gradient descent:

$$v_{\text{cat}} \leftarrow v_{\text{cat}} - \eta \frac{\partial \mathcal{L}}{\partial v_{\text{cat}}} \quad v_{\text{walks}} \leftarrow v_{\text{walks}} - \eta \frac{\partial \mathcal{L}}{\partial v_{\text{walks}}}$$

The dot product changes indirectly because the vectors change
 $v_{\text{cat}} \cdot v_{\text{walks}}$ is modified through these updates.

Interpretation

- If the gradient encourages alignment, the dot product increases.
- If the gradient discourages alignment, the dot product decreases.
- The change happens because lowering the loss reshapes the geometry.

Properties of self-attention

Take away

Dot products increase or decrease because gradient descent mechanically adjust them in whatever direction to lower the loss.

Properties

- The attention weights are computed dynamically from the input.
- The dot-product + softmax operation has no parameters.
- Without positional encoding, self-attention is permutation invariant.

Basic views of an embedding in self-attention

Fundamental idea

A token simultaneously asks a question (query), advertises what it contains (key), and carries content (value).

Three roles per token in a sequence

- Query q_i : What am I looking for in other tokens?
- Key k_j : What do I offer that other tokens might look for?
- Value v_j : If someone attends to me, what information do I provide?

Self-attention mechanism in transformers

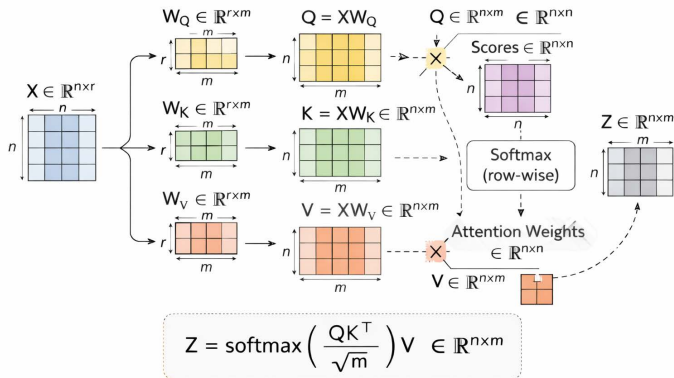


Figure: Self-attention mechanism in Transformers

Basic views of an embedding in self-attention-II

Learned projections and attention weights

Embeddings are projected using learned matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Attention weights are computed as:

$$w_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right)$$

The output representation becomes:

$$y_i = \sum_j w_{ij} v_j$$

Self-attention mechanism in transformers

Single-head self-attention (symbolic dimensions)

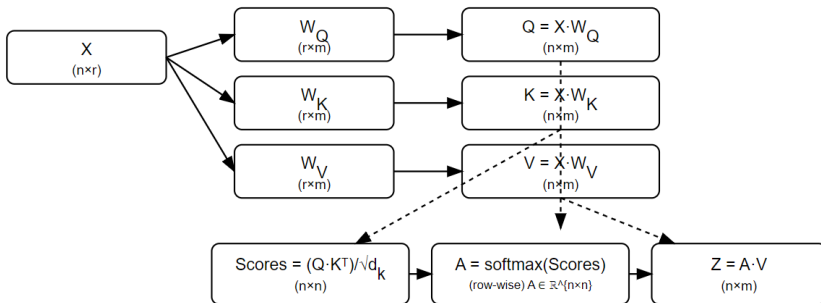


Figure: Self-attention mechanism in Transformers

Toy example (follow up)

"The cat walks on street":

- Let's take embeddings as 2D vectors

	words in sequence	input vector	embeddings
1	the	x_{the}	(0.1, 0.0)
2	cat	x_{cat}	(1.0, 0.9)
3	walks	x_{walks}	(0.9, 1.0)
4	on	x_{on}	(0.0, 0.1)
5	street	x_{street}	(0.8, 0.7)

- Projection matrices W_Q , W_K , W_V , we assume the following numeric values:

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad W_K = \begin{pmatrix} 0.8 & 0.2 \\ 0.1 & 0.9 \end{pmatrix} \quad W_V = \begin{pmatrix} 1 & -0.3 \\ 0.2 & 0.7 \end{pmatrix}$$

Toy example-II

$$q_i = W_Q x_i$$

$$q_{\text{the}} = (0.1, 0.0)$$

$$q_{\text{cat}} = (1.0, 0.9)$$

$$q_{\text{walks}} = (0.9, 1.0)$$

$$q_{\text{on}} = (0.0, 0.1)$$

$$q_{\text{street}} = (0.8, 0.7)$$

$$k_i = W_K x_i$$

$$k_{\text{the}} = (0.08, 0.01)$$

$$k_{\text{cat}} = (0.98, 0.91)$$

$$k_{\text{walks}} = (0.92, 0.91)$$

$$k_{\text{on}} = (0.02, 0.09)$$

$$k_{\text{street}} = (0.78, 0.71)$$

$$v_i = W_V x_i$$

$$v_{\text{the}} = (0.1, 0.02)$$

$$v_{\text{cat}} = (0.73, 0.83)$$

$$v_{\text{walks}} = (0.6, 0.88)$$

$$v_{\text{on}} = (-0.03, 0.07)$$

$$v_{\text{street}} = (0.59, 0.65)$$

Scaled dot-product scores for token `cat`

$$s_{\text{cat},j} = \frac{q_{\text{cat}}^\top k_j}{\sqrt{2}}$$

Computed values

$$s_{\text{cat},\text{the}} = 0.06293$$

$$s_{\text{cat},\text{cat}} = 1.27209$$

$$s_{\text{cat},\text{walks}} = 1.28057$$

$$s_{\text{cat},\text{on}} = 0.07142$$

$$s_{\text{cat},\text{street}} = 1.00338$$

Softmax normalization for token cat

$$\sum_j e^{s_{\text{cat},j}} = 13.098$$

$$w_{\text{cat},j} = \frac{e^{s_{\text{cat},j}}}{\sum_k e^{s_{\text{cat},k}}}$$

Attention weights

$$w_{\text{cat},\text{the}} = 0.0813$$

$$w_{\text{cat},\text{cat}} = 0.27248$$

$$w_{\text{cat},\text{walks}} = 0.27469$$

$$w_{\text{cat},\text{on}} = 0.081997$$

$$w_{\text{cat},\text{street}} = 0.208199$$

$$\sum_j w_{\text{cat},j} = 1$$

$$y_{\text{cat}} = \sum_j w_{\text{cat},j} v_j = (0.50037, 0.61223)$$

The output is a weighted combination of value vectors, not a probability vector.

Basic Self-Attention (No Projections)

```
1 import torch
2 import torch.nn.functional as F
3
4 # x has shape (batch_size, seq_len, d_model)
5 x = ...
6
7 # Compute raw attention scores: (b, t, t)
8 raw_scores = torch.bmm(x, x.transpose(1, 2))
9
10 # Optional scaling (recommended)
11 d = x.size(-1)
12 raw_scores = raw_scores / d**0.5
13
14 # Normalize across keys (last dimension)
15 weights = F.softmax(raw_scores, dim=2)
16
17 # Weighted sum of values (here values = x)
18 # Output shape: (b, t, d_model)
19 y = torch.bmm(weights, x)
```

Scaled Dot-Product Attention (with Projections)

```
1 import torch
2 import torch.nn.functional as F
3 from math import sqrt
4 # x: (batch_size, seq_len, d_model)
5 # Linear projections
6 Q = x @ W_Q      # (b, t, d_k)
7 K = x @ W_K      # (b, t, d_k)
8 V = x @ W_V      # (b, t, d_v)
9
10 # Scaled dot-product attention
11 scores = torch.bmm(Q, K.transpose(1, 2)) / sqrt(d_k)
12 # scores: (b, t, t)
13 weights = F.softmax(scores, dim=2)
14 # weights: (b, t, t)
15 # Weighted sum of values
16 y = torch.bmm(weights, V)
17 # y: (b, t, d_v)
```

Final Takeaway: Structure of Self-Attention

Input to attention

x_i : Original embedding (or output of the previous layer).

Who attends to whom

$$q_i = W_Q x_i, \quad k_j = W_K x_j$$

Similarity score:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

Attention weights:

$$w_{ij} = \text{softmax}(s_{ij})$$

Final Takeaway: Value Aggregation

What information is aggregated

$$v_j = W_V x_j$$

Output vector:

$$y_i = \sum_j w_{ij} v_j$$

Geometric interpretation

- The weights satisfy $w_{ij} \geq 0$ and $\sum_j w_{ij} = 1$.
- Therefore, y_i is a convex combination of value vectors.
- Attention redistributes existing information — it does not create new directions.

References

- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).
- Bloem, P. (2019). *Transformers from Scratch*.
<https://peterbloem.nl/blog/transformers> (Figures adapted from Bloem (2019), with modifications.)
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.
- PyTorch Documentation.
<https://pytorch.org/docs/stable/>